ALGORITHM DESIGN • UNIT 3

DYNAMIC PROGRAMMING

A study of efficient problem-solving using stored subproblems

Name – Chandni Amesar

PRN – 25070521218

Semester – 3

Section – D

Generated using ChatGPT

1. What is Dynamic Programming?

Solve smaller problems once, store their answers, and reuse them.

The core idea

Dynamic Programming (DP) is a technique for solving problems by breaking them into smaller subproblems and storing their results so the same work is not repeated.

Overlapping subproblems

The same smaller problem appears again and again.

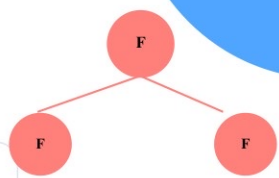
Optimal substructure

An optimal solution can be built from optimal solutions to smaller parts.

State + transition

Define what each DP cell means, then define how it is computed.

WHY DP?



Naive recursion repeats work.

DP stores the answer

2. Two Ways to Build a DP Solution

Both approaches avoid repeated computation; they differ in direction.

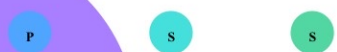
TOP-DOWN • MEMOIZATION

Start with the original problem and recursively break it down.

✓ If a state is already solved → reuse it.
✓ If not → compute it and store it.

Best when only a fraction of states are needed.

Original → smaller → memoized result



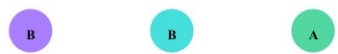
BOTTOM-UP • TABULATION

Start from the smallest known states and build upward using a table.

✓ No recursion stack.
✓ Easy to see the full DP table.

Best when most states are needed.

Base cases → table → final answer



3. How a DP Solution is Designed

A repeatable 5-step workflow you can use in many problems.

1	**Define the state**	What does dp[i] or dp[i][j] represent?
2	**Find base cases**	What are the smallest / simplest answers?
3	**Write the transition**	How does the current state depend on earlier states?
4	**Choose an order**	Top-down recursion or bottom-up table?
5	**Read the answer**	Which cell/state contains the final optimum?

dp[state] = best answer
for this state

4. Main Example: 0/1 Knapsack

Each item is either completely selected (1) or completely skipped (0).

Problem

Capacity = 4

Item 1 → value 1, weight 4
Item 2 → value 2, weight 5
Item 3 → value 3, weight 1

Goal: maximize total value without exceeding capacity.

Answer: choose Item 3 → value = 3.

ITEM 1	ITEM 2	ITEM 3
Value 1	**Value 2**	**Value 3**
Weight 4	Weight 5	Weight 1

At every item, ask:



Take it only if its weight fits the remaining capacity.

5. 0/1 Knapsack — DP Table Walkthrough

dp[i][w] = maximum value using the first i items with capacity w.

i\w	0	1	2	3	4
0	0	0	0	0	
1	0	0	0	1	
2	0	0	0	1	
3	0	3	3	3	

```
if weight[i-1] <= w:
  dp[i][w] = max(
     value[i-1] + dp[i-1][w-weight[i-1]],
     dp[i-1][w]
  )
else: dp[i][w] = dp[i-1][w]
```

Why is the final answer 3?

- Item 1 fits at capacity 4, giving value 1.
- Item 2 is too heavy (weight 5).
- Item 3 fits and gives value 3 → best.

Time

O(n × W)

Space

O(n × W)
(can be optimized)

6. Where Dynamic Programming is Useful

DP appears whenever the same choices/subproblems repeat.

Knapsack

Budget / capacity allocation

Shortest paths

Best route under constraints

Sequence problems

LCS, edit distance, alignment

Scheduling

Choose the best compatible jobs

Finance

Portfolio / resource optimization

Bioinformatics

Compare and align biological sequences

Strength: avoids repeated work • Limitation: can use significant memory

7. Advantages, Limitations & Takeaway

A quick comparison to close the story.

ADVANTAGES

✓ Faster than naive recursion
✓ Reuses computed results
✓ Gives optimal solutions for suitable problems
✓ Works well with many optimization tasks

LIMITATIO

• Extra memory r
• State/transition
• Not every probl

THE DP MINDSET



Key takeaway: define the state clearly, write the transition carefully, and let the table do the repeated work.